Ch1: OS Goals, Operation, Evolution

User view: ease of use, abstraction, powerful instr, easy run programs.
System view: resource allocator, control program.
OS goals: execute user progs; make solving easier; convenient; efficient HW.
Multiprogramming: multiple jobs in mem; switch on I/O wait; max CPU throughput.
Batch: one job at a time; no interactivity.
Time-sharing: frequent switches; interactive; min response time.
Protection between user procs: separate address spaces; dual-mode; privileged instr; timer.
OS protection: dual-mode; syscalls/interrupts change mode; timer; memory protection; user IDs.
Storage hierarchy: access time: registers(compiler) < cache(hardware) < disk. Size: registers < cache < disk. (all else os)
Architected registers limited by: ISA, instruction format, hardware cost, compiler needs.
Cache size restricted by: cost, physical space, access speed, power/heat, yield, diminishing returns.
Multiple cache levels: balance speed/capacity/cost; exploit locality.
T/F: (a) F – compiler manages arch regs. (b) F – OS manages disk; user uses file system.
OS virtualization: single CPU→multiple CPUs; RAM→virtual memory; disk→file system.

Ch2: OS Services, Syscalls, Structures

User services: UI, program exec, I/O, file system, IPC, error detection.
Efficient operation: resource allocation, accounting, protection/security.
System calls: programming interface to OS; request privileged service; trap; mode switch.
Function calls: user mode; no privilege; low overhead.
API: high-level, simpler, portable. Prefer API over direct syscall.
Register arg limitations: limited number. Overcome: block/table on stack; pass stack loc in reg; stack push/pop.
Compiler: source→assembly. **Assembler:** assembly→object. **Linker:** object+libs→executable. **Loader:** loads into memory.

Ch3: Processes, Scheduling, IPC

Process: program in execution, active. **Program:** passive, text.
Address space: text (program bi), data (global and static data), heap (dynamically allocated data), stack (local vars).
Process states: New→Ready→Running→Terminated.
Transitions: interrupt: Running→Ready; dispatch: Ready→Running; I/O completion: Waiting→Ready; I/O wait: Running→Waiting.
Privileged instr: set timer, read clock (usually), switch user→OS, turn off interrupts. Not shared mem set.
Context switch: save PCB/regs/PC; restore next. Benefit: multitasking. Drawback: overhead.
fork: create child. **exec:** replace memory with new program.
Code: `pid=fork(); if(pid==0) execlp("/bin/ps","ps",NULL); else wait(NULL);`
Shared mem vs msg passing: shared fast, convenient, needs sync; msg slower, easier distributed, no conflict.
Pipe sync code: use pipe to make parent wait for child; shared_num not shared after fork; final 0 if parent waits.
Pipes adv: simple, FIFO. Limits: related processes, unidirectional, no network. FIFOs, msg queues, shared mem, sockets fix.
Socket server: socket, bind, listen, accept, recv/send, close.
Client: socket, connect, send/recv, close.

Ch4: Threads – User vs Kernel Level

Two threads addr space: shared text, data, heap; separate stacks/regs.
Two processes: separate everything.
Threads adv: less overhead, sharing, responsiveness.
Disadv: less protection, sync issues.
User vs kernel threads:
User: fast, flexible; blocking blocks all; no SMP.
Kernel: blocking doesn't block all; uses SMP; slower overhead.
Multithreading models: many-to-one - one kernel thread Disad: one blocking call blocks all , one-to-one: each user thread -> one kernel thread, many-to-many: m user threads -> n kernel threads (brief).

Consider the following program:

```
int main(){
   pid_t pid;

   printf("Begin program\n");
   pid = fork();    /* fork another process */
   if (pid == 0) { /* child process */
      execlp("/bin/ls", "ls", NULL);
      printf("Process exiting: %d\n", getpid());
   }
   if (pid > 0) { /* parent process */
      wait(NULL);
      printf ("Process exiting: %d\n", getpid());
   }
   exit(0);
}
```

Assume that,
(a) the "fork" and "exec" calls are successful,
(b) the parent process has the process id=10, and child process has process id=11.

Questions:

1. The child process is *guaranteed* to finish before the parent -- True
2. The line "Process exiting: 11" is printed only once -- False
3. The line "Begin program" is printed only once -- True

File desc table: stdin, stdout, stderror, fds[0] (read), fds[1] (write)

Int fds[2]; // creates an array of values for file descriptors
pipe(fds); fills the file desc table, starting with 1st available
read(fds[0], buf, 5); // read directly from the read end
write(fds[1], "hello", 5); // write directly to the write end
dup2(oldfd, newfd); // should use for stdin and stdout, more robust
close(index); // closes the given file descriptor

Create: mkfifo("filename", permissions); — creates the special file.

Open: open("filename", O_RDONLY); or open("filename", O_WRONLY); — gets a file descriptor (just like a regular file).

Read/Write: read(fd, buf, size); or write(fd, buf, size);

Close: close(fd);

Delete (optional): unlink("filename"); — removes the FIFO from the filesystem.

shmget()	Get (create or find) a shared memory segment
shmat()	Attach the segment to the process's address space
shmdt()	Detach the segment from the process's address space
shmctl()	Control operations (e.g., delete the segment)

c Copy Download

```c
int segment_id;
char *shared_memory;
const int size = 4096;

// 1. ALLOCATE a shared memory segment
segment_id = shmget(IPC_PRIVATE, size, S_IRUSR|S_IWUSR);

// 2. ATTACH it to this process's address space
shared_memory = (char *) shmat(segment_id, NULL, 0);

// 3. USE it like normal memory
sprintf(shared_memory, "EECS 678");
printf("%s\n", shared_memory);

// 4. DETACH it
shmdt(shared_memory);

// 5. DELETE it
shmctl(segment_id, IPC_RMID, NULL);
```

syscall_overhead.c | pthreads.c 2

pthreads.c > main(int, char * [])

```c
#include <pthread.h>
#include <stdio.h>

int sum; /* data shared by all threads */
void *runner(void *param); /* thread function prototype */

int main (int argc, char *argv[])
{
    pthread_t tid; /* thread identifier */
    pthread_attr_t attr /* set of thread attributes */

    if(atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >=0\n", atoi(argv[1]));
        return -1;
    }
    /* get the default thread attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL); // blocks until runner thread exits
    fprintf(stdout, "sum = %d\n", sum);
}
/* The thread will begin control in this function */
void *runner (void  *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for(i=1 ; i<=upper ; i++)
        sum += i;

    pthread_exit(0);
}
```

pthread_create(&tid, &attr, start_routine, arg)	Create a new thread. tid gets the thread ID.
pthread_join(tid, &retval)	Wait for a thread to terminate; retrieves return value.
pthread_exit(retval)	Terminate the calling thread.
pthread_self()	Return the calling thread's ID.
pthread_equal(t1, t2)	Compare two thread IDs.
pthread_detach(tid)	Detach a thread so its resources are automatically reclaimed on exit.
pthread_cancel(tid)	Request cancellation of another thread.
pthread_attr_init(&attr)	Initialize thread attributes with defaults.
pthread_attr_destroy(&attr)	Destroy attribute object.